

CLASE 3

Proyecto Django, URLs y Views

Convertiremos una ruta del navegador en una respuesta del servidor

URL → View → Response

EcoEnergy necesita responder dos solicitudes

- GET / debe presentar el nombre del sistema.
- GET /zonas/3/dispositivos/ debe reconocer la zona solicitada.
- Una zona inexistente debe responder 404.

Cada ruta necesita una View capaz de construir una respuesta HTTP coherente.

¿Cómo sabrá Django qué función ejecutar para cada dirección?

Los archivos `urls.py` comparan la ruta solicitada y dirigen la petición hacia la View correspondiente.

Hoy construiremos el primer recorrido ejecutable

OBJETIVO DE APRENDIZAJE

Crear un proyecto y una aplicación Django, configurar URLs e implementar Views que entreguen respuestas HTTP básicas y verificables.

Al finalizar podrás...

- 01 Diferenciar proyecto y aplicación.
- 02 Conectar URLconf del proyecto y de la aplicación.
- 03 Construir respuestas 200 y 404.
- 04 Comprobar y registrar el funcionamiento.

Antes de programar, confirma el punto de partida

```
git status

source .venv/Scripts/activate

python -c "import sys; print(sys.executable)"

python -m django --version

python -m pip check
```

Observa

- Trabaja dentro del repositorio correcto.
- El intérprete debe pertenecer a .venv.
- Django debe responder desde ese entorno.
- Las dependencias no deben presentar conflictos.

El proyecto configura; la aplicación resuelve una función

PROYECTO · config

- Configuración global.
- URLs principales.
- Arranque y despliegue.
- Integra varias aplicaciones.

APLICACIÓN · dispositivos

- Funcionalidad específica.
- Views y URLs propias.
- Modelos y Templates futuros.
- Puede reutilizarse o evolucionar.

Un proyecto Django organiza el sistema; cada aplicación concentra una responsabilidad del dominio.

Paso 1: crea el proyecto y la aplicación

```
python -m django startproject config .
```

```
python manage.py startapp dispositivos
```

```
python manage.py check
```

```
# Ejecutar el servidor
```

```
python manage.py runserver
```

Observa

- El punto crea config dentro del repositorio actual.
- manage.py administra este proyecto.
- startapp crea un módulo funcional.
- check valida la configuración conocida.

Paso 2: registra la aplicación en settings.py

```
# config/settings.py

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "dispositivos",
]

# Verificación
python manage.py check
```



Observa

- INSTALLED_APPS declara módulos del proyecto.
- El nombre debe coincidir con la aplicación.
- check detecta errores de configuración.

Paso 3: una View recibe request y construye response

```
# dispositivos/views.py
from django.http import HttpResponse

def inicio(request):
    return HttpResponse(
        "<h1>EcoEnergy</h1>"
        "<p>Back End en funcionamiento</p>"
    )
```

Observa

- request representa la solicitud recibida.
- request representa **la solicitud HTTP que llegó desde el cliente** —normalmente el navegador— hasta la View.
- La función contiene la coordinación inicial.
- HttpResponse construye la respuesta HTTP.

Paso 4: urls.py conecta la ruta con la View

```
# dispositivos/urls.py
from django.urls import path
from . import views

app_name = "dispositivos"

urlpatterns = [
    path("", views.inicio, name="inicio"),
]
```

Observa

- path compara la ruta solicitada.
- views.inicio indica qué función ejecutar.
- name identifica la ruta dentro de Django.
- app_name evita colisiones entre aplicaciones.

Paso 5: el proyecto delega rutas con include

```
# config/urls.py
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("dispositivos.urls")),
]

# Probar
python manage.py runserver
```

Observa

- config/urls.py recibe la solicitud primero.
- include delega rutas a la aplicación.
- La ruta vacía conserva / como inicio.

Paso 6: una ruta dinámica entrega datos a la View

```
# dispositivos/views.py
def dispositivos_zona(request, zona_id):
    if zona_id != 3:
        return HttpResponse(
            "Zona no encontrada", status=404
        )
    return HttpResponse(
        f"Dispositivos de la zona {zona_id}"
    )

# dispositivos/urls.py
path(
    "zonas/<int:zona_id>/dispositivos/",
    views.dispositivos_zona,
    name="por_zona",
)
```

Observa

- <int:zona_id> captura un número desde la URL.
- Django entrega zona_id como argumento.
- La View decide entre 200 y 404.

El mensaje de error indica dónde comenzar

SEÑAL

- 404 en una URL esperada.
- NameError o ImportError.
- La View no recibe zona_id.
- El servidor no inicia.

PRIMERA COMPROBACIÓN

- Ruta escrita y urlpatterns.
- Importaciones y nombres.
- Convertidor y parámetros.
- Última línea del traceback.

Comprueba el recorrido en orden: URL principal → URL de aplicación → View → Response.

Laboratorio: crea una ruta propia y verificable

SITUACIÓN

Implementa una segunda funcionalidad en tu Proyecto Integrado o en EcoEnergy sin copiar exactamente la demostración.

1

**Elige un recurso:
dispositivo, zona o
entidad equivalente.**

2

**Crea una View con
un parámetro
entero.**

3

**Registra la ruta en
urls.py de la
aplicación.**

4

**Prueba un caso 200
y un caso 404.**

Entrega individual: dos URLs probadas, estados observados y enlace al commit.

Ticket de salida: explica tu primera respuesta Django

TICKET DE SALIDA

¿Qué archivos recorre GET /zonas/3/dispositivos/ y dónde se decide responder 200 o 404?

Responde brevemente

1. Respuesta individual en cuatro pasos + enlace al commit.

PRÓXIMA CLASE

Clase 4: Templates, herencia y datos enviados desde una View.